

סיכום תרגולים 2026 – מערכות הפעלה

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

הערה. לפי המרצה, התרגולים מכסים לרוב חומר שונה לגמרי מהחומר בהרצאה ואף נשאלים במבחן בחלק ייעודי שנקרא חומר מהתרגול, לכן הסיכום הנ"ל מפוצל מסיכום ההרצאה.

תוכן עניינים:

2	תרגול 1: מבוא למערכות הפעלה, system calls, לינוקס
5	תרגול 2: bash
8	תרגול 3: fork, exec, exit
12	תרגול 4: Threads, mutex
17	תרגול 5: קבצים
21	תרגול 6: semaphores
24	תרגול 7: signals

תרגול 1

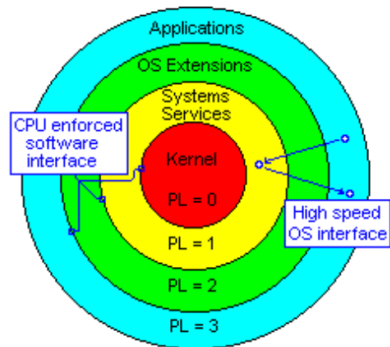
מערכת ההפעלה

מערכת ההפעלה היא **תוכנת מחשב**. התפקיד שלה בעיקר לשמש כמתווכת בין משתמש לחומרה, נותנת שירות לתוכניות ולמשתמשים, מנהלת הרצה של שאר התוכניות ומנהלת את המשאבים של התוכנית (זיכרון, דיסק, מעבד וכו'). ביתר פירוט תפקידיה הם:

- **ניהול זיכרון:** ניהול הקצאה דינאמית של זיכרון כיוון שאלו דברים שמשתנים כל הזמן, מחליטה אילו חלקים של תהליכים לטעון לזיכרון ואילו לא, שומרת מידע על כל פרוסס ומנהלת זיכרון וירטואלי (נדון בהמשך). משתמשת גם בהארד דיסק.
- **ניהול קבצים:** קבצים הינם יחידת אחסון לוגית של מידע, ניהול הרשאות גישה לקבצים, יצירת היררכיה ע"י תיקיות, וכל הפעולות שקשורות לקבצים – יצירה, קריאה, מחיקה וכו'.
- **אחריות כלפי החומרה:** הגנה על החומרה, כיצד? באמצעות מתן גישה מאוד מבוקרת לחומרה. אם היינו יודעים כי רק תוכנית אחת רצה, או שבתוכנית מראש אין שגיאות לא היינו צריכים מערכת הפעלה. אבל אלו כמובן לא הנחות לגיטימיות, בטח לא כיום. כמו כן, **היא צריכה להגן על המעבד:** באמצעות שני מצבי ריצה – *kernel and user modes*. כמו כן, מנגנון הפסיקות (interrupt) מאפשר למערכת ההפעלה להשתלט על המעבד, המעבד עובר לשגרת פסיקה ועובר לקרנל מוד. כמו כן, היא מספקת ממשקים אחידים לחומרה. תפקיד נוסף כלפי החומרה – לנהל וליעל אותה בצורה המרבית (נרצה להשתמש בה כמה שיותר, ונרחיב על כך בהמשך בהרצאות: למשל אנחנו נרצה כי היא תשתמש בכמה שפחות זמן מעבד).

ארכיטקטורת פון נוימן:

איך מחשב פועל?



- מעבד (CPU): אחראי לביצוע התוכנית, תפקידו לבצע את הפקודות (שפת מכונה) המאוכסנות בזיכרון של המחשב. נשאלת השאלה – האם הוא יבצע כל פקודה שנבקש? זה תלוי במושג שנקרא "רמת הפריווילגיה" (Rings) שניתנה לפקודה הנוכחית. כאשר קוד מסוים מבקש לגשת לאזור בזיכרון, החומרה מאשרת את הגישה בהתאם במגבלות הבאות: לפי סוג הפקודה, ולפי *privilege* של המבקש (לכל אזור בזיכרון מוגדר DPL), הגישה מותרת רק אם הפריווילגיה של המבקש קטנה מהDPL. לצורך המחשה (משמאל): אם הקרנל שלנו רוצה להריץ *word*, הPL של הקרנל הוא 0 ושל *word* הוא 3 וכיוון ש $3 > 0$ אזי הוא יכול לבצע.

בכל רגע נתון תוכנית אחת בלבד רצה על המעבד. פסיקה (interrupt) גורמת למעבד להפסיק את ריצתו והשליטה עוברת למערכת ההפעלה – היא נשלחת כשרוצים לבצע קריאת מערכת, שיש שגיאה וכו'.

- זיכרון: מאחסן את הפקודות לביצוע והנתונים שהתוכנית משתמשת בהם.
- רגיסטרים – תאי אחסון נתונים אשר כל הפעולות האריתמטיות והלוגיות מתבצעות עליהם.

PSW: מדובר ברגיסטר שאחד מתפקידיו הוא להבחין אם אנחנו ב *User or kernel mode*, אם ביט מסוים באוגר זה דולק אזי התהליך שרץ חוסם פעולות מסוימות כמו IO, גישה לכל חלקי הזיכרון ועדכון טבלאות/רגיסטרים מסוימים. אם הביט כבוי – יש גישה להכל.

חשוב לשים לב ולזכור: ככל שהRing קטן יותר – רמת ההרשאה גבוהה יותר.

System call: איך תהליך בעל *privilege* גבוה יכול לבצע פעולות שדורשות *privilege* נמוך? ע"י *system call*: מבקשים ממערכת ההפעלה שתסייע. מערכת ההפעלה דואגת לטפל בבקשה ולהחזיר את הנתונים המבוקשים למבקש הבקשה.

Linux ופקודות שימושיות בלינוקס

לינוקס היא מערכת הפעלה שנבנתה בהשראת מערכת Unix. היא חופשית וחינמית, ניתנת להורדה בחינם מהאינטרנט. ודוגלת בגישה של Open Source: קוד פתוח, הקוד פתוח לכולם ומפותח בשיתוף פעולה ע"י אלפי מתכנתים ברחבי העולם (זהו פרויקט בכלל, שהחל ב-1991 ע"י סטודנט), כלומר כל אחד יכול לראות את קוד המקור, לקרוא אותו ולראות בדיוק מה התוכנה עושה. המערכת רצה על כמעט כל סוג מעבד, מאפשרת למספר משתמשים להיות מחוברים למערכת בו זמנית, ותומכת בריבוי תהליכים.

פקודות שימושיות:

- **Man:** פקודה להצגת דפי הסבר על פקודות המערכת השונות. כאשר נרצה לדעת מה פקודה מסוימת עושה ואילו דגלים יש לה נכתוב `man` ואז את שם הפקודה. למשל: `man ls` יציג את דף ההסבר של הפקודה `ls`, ונוכל לראות מה עושים דגלים כמו `-a` וכו'. כמו כן, לפעמים לא נזכור את שם הפקודה המדויק, אך יודע מה אתה רוצה לעשות. לכן ניתן לחפש באמצעותה לפי נושא. כך: `<topic> -k man`, הדגל `-k` מבצע חיפוש במאגר המדריכים.
- דפי המדריך אינם סתם רשימה אחת ארוכה, אלא ספרייה שמחולקת לפי נושאים. זה עוזר לנו לדעת אם אנחנו מסתכלים על פקודת טרמינל או פקודת C. לכן, ישנם סקשנים שונים:
 1. `Section 1-commands`: פקודות משתמש רגילות כמו `cd`, `ls` וכו'.
 2. `Section 2- system calls`: רשימה של קריאות המערכת
 3. `Section 3- C library functions`: פונקציה של ספריות סטנדרטיות כמו `printf`, `scanf` וכו'.ויש עוד... ישנם סה"כ 9 סקשנים.
- **Ls (list directory contents):** פקודה זו מדפיסה את התוכן של ספרייה מסוימת. במידה ונריץ את הפקודה ללא פרמטרים, היא תריץ את תוכנה של הספרייה הנוכחית. אם נוסיף דגלים כמו `a`, למשל `ls -al` יודפסו גם שמות הקבצים הנסתרים ונקבל פלט מורחב עם נתונים נוספים על הקבצים ותתי הספריות.
- **Pwd:** `print name of current/working directory`, מדפיסה את נתיב הספרייה הנוכחית.
- **Cd:** משמשת כמעבר מספרייה לספרייה, ישנן הרחבות: למשל `cd ..` היא מעבר מהספרייה הנוכחית לזו שמכילה אותה, `cd -` כחזרה לספרייה הקודמת שהיינו בה, `cd ~` מעבר לספריית הבית של המשתמש.
- **Mkdir:** יצירת ספרייה חדשה.
- **Rmdir:** מחיקת ספרייה (ריקה!), אם לא ריקה יהיה שגיאה.
- **Mv:** העברת קובץ. למשל: `mv myfile backups/` למעבר הקובץ `myfile` אל הספרייה `backups`.
- **Cp:** פקודה שמעתיקה קבצים, דוגמאות:
העתקת הקובץ `myfile` לתוך הספרייה `backups`: `cp myfile backup/`
העתקת כל הקבצים בעלי הסיומת `txt` לתוך הספרייה `txts`: `cp *.txt /home/op/txts`
- **Chmod:** שינוי הרשאות לקובץ. למשל: שינוי הרשאות לקובץ כך שכל מי שנוגע יוכל להריץ, לקרוא ולכתוב: `chmod 777 myfile`.
- **Cat:** פקודה שמשרשרת תוכן של קבצים ומדפיסה אותם למסך, למשל: הדפסת הקובץ `myfile` למסך ע"י `cat myfile`. ניתן גם להדפיס קובץ אל תוך קובץ אחר: `cat myfile > yourfile`
- **Gcc:** קומפיילר עבור השפות C, C++. למשל קמפול הקובץ: `gcc myprog.c`, שם קובץ ההרצה שיווצר יהיה `a.out`. כמו כן ניתן גם לקמפל ולתת את שם קובץ ההרצה בעצמנו ע"י `gcc myprog.c -o myprog`.

• הרשאות, אבטחה וקבוצות בLinux

לינוקס היא מערכת Multi-user, כלומר כמה משתמשים יכולים להיות מחוברים למערכת במקביל (או בזמנים שונים). לכל משתמש יש userID, מס' ייחודי משלו. באמצעות הפקודה who ניתן לראות מיהם המשתמשים שמחוברים כרגע, כולל התאריך שבו התחבר, מהיכן הוא מחובר וכו'. לינוקס שומרת את פרטי המשתמשים במערכת באמצעות קובץ בשם etc/passwd בו כל שורה בקובץ מתארת משתמש אחד, ומחולקת ל 7 שדות שמופרדים בנקודתיים: שם המשתמש, סיסמה (מסומנת באיקס בפועל, היא מאובטחת ושמורה בקובץ אחר), USERID, GROUPID, וכו'.

קבוצות: כל משתמש יכול להיות שייך למספר קבוצות (יש לו קבוצת ברירת מחדל אליה שייך), כדי לבדוק באילו קבוצות נמצא המשתמש מסוים נוכל להריץ `groups <username>`. משתמש יכול להחליף גם את קבוצת ברירת המחדל שלו ע"י שימוש בפקודה `newgrp <groupName>`. הקובץ `etc/group` מכיל רשימה של כל הקבוצות במערכת. לכל קבוצה נשמרת רשומה בקובץ עם הפרטים הבאים: שם קבוצה, סיסמה, ID, משתמשים שבקבוצה.

מדוע שנשתמש בקבוצות? זו הדרך של המערכת לנהל הרשאות בצורה יעילה – במקום לתת לכל משתמש הרשאות באופן פרטני, היא נותנת פר קבוצה וכל קבוצה מקבלת מפתח למשאבים מסוימים.

ישנן 3 סוגי הרשאות: `read,write,execute`. ההרשאות מסומנות ע"י מספר `execute=1,read=4,write=2`. פירוש ההרשאה 7 למשל: $4+2+1$, כלומר הרשאות כתיבה, קריאה וביצוע.

לכל קובץ או תיקייה במערכת יש כרטיסיית הרשאות שמתייחסת לשלושת הישיות הללו:

- **User:** בעל הקובץ. זהו המשתמש שיצר את הקובץ, ולו לרוב יש את השליטה המלאה – יכול לקרוא לכתוב ולבצע. הוא היחיד פרט לroot שיכול לשנות את ההרשאות של הקובץ לאחרים.
- **Group:** הקבוצה המשויתת. לכל קובץ משויכת קבוצה אחת מסוימת, כל מי שחבר בקבוצה זו יקבל את ההרשאות שהוגדרו עבורו.
- **Other:** כל היתר, כל משתמש במערכת שלא נכנס תחת 2 ההגדרות הקודמות. ההרשאות כאן יהיו הכי מצומצמות.

User Mask: מדובר במסכה שחלה על תהליכים במערכת, כשתוכנה יוצרת קובץ חדש `umask` מגביל את ההרשאות שלו באופן אוטומטי על מנת למנוע מצב שקובץ נוצר עם הרשאות רחבות מדי. הפקודה מחזירה מספר שמסמל את הmask של התהליך שהריץ את הפקודה.

כיצד משנים סיסמה? על מנת לשנות סיסמה, המערכת חייבת לכתוב את הסיסמה החדשה לתוך הקובץ של המערכת שכפי שאמרנו `etc/passwd/`. הקובץ הנ"ל נמצא בבעלות root ואין למשתמש רגיל הרשאות של כתיבה אליו. **אז איך הסיסמה תעודכן?**

בהרשאות ישנה האות s, זוהי הרשאה מיוחדת שנקראת SetUID. כל מי שיש לו הרשאת גישה על הקובץ (למשל, המשתמש הרגיל) יריץ את התוכנה לא עם ההרשאות שלו, אלא עם ההרשאות של יוצר התוכנה (במקרה של הסיסמה היוצר הוא root). זה מה שמאפשר לקבל כוחות של root לזמן קצר שהפקודה הזו רצה. כיצד קובעים הרשאה זו? באמצעות `chmod` ומוסיפים ספרה רביעית לפני שלוש הספרות הרגילות, כאשר הספרה 4 קובעת את s עבור uid (משתמש), הספרה 2 עבור הקבוצה והספרה 6 עבור שניהם.

תרגול 2

מהו shell? מדובר בתוכנית שמאפשרת ממשק לפקודות בין משתמש יחיד לבין מערכת ההפעלה (kernel). ישנן 2 משפחות של shell:

- Sh: סינטקס שמזכיר פסקל, הגרסה הכי פופולרית היא bash.
- Csh: סינטקס שמזכיר C, הגרסה הכי פופולרית היא tcsh.

ניתן לבדוק מהו shell שאנו משתמשים בו באמצעות הפקודות `echo $SHELL`.

בכל תהליך, קיימים **שלושה** ערוצים סטנדרטיים פתוחים שדרכם עובר המידע. כל ערוץ שכזה מיוצג ע"י `fd`:

1. `Fd 0 (stdin)`: ערוץ קלט סטנדרטי, משם התוכנית קוראת נתונים.
2. `Fd 1 (stdout)`: ערוץ פלט סטנדרטי, לשם התוכנית כותבת.
3. `Fd 2 (stderr)`: ערוץ שגיאות סטנדרטי, הוא מחובר למסך כברירת מחדל, הוא נפרד למען הודעות שגיאה דחופות.

על פקודות ושכדאי לדעת:

- אם נעשה `ls > a.txt` (באשר `a.txt` הוא קובץ כלשהו, נכתוב לתוכו למעשה את שמות כל הקבצים בתיקייה הנוכחית). – אם הקובץ הנ"ל `a.txt` לא היה קיים, shell יצר אותו לאחר פקודה זו והכניס אליו את שמות הקבצים.
- `>`: הפניית פלט עם דריסה. למשל, אם נעשה `a.txt > echo "Hello!"` אז לא משנה מה היה קודם בקובץ `a.txt` כעת יהיה בו "Hello" בלבד.
- `>>`: הפניית פלט ללא דריסה, עם הוספה.
- `<`: הפניית קלט מקובץ לפקודה. למשל: `sort < a.txt` ייקח את כל הערכים שבקובץ ויעביר אותם אל פונקציית `sort` שתמיין אותם.
- `<<`: מאפשר כמה שורות קלט שנרצה לכתוב עד שנגיע ל`delimiter`. דוגמה:

```
Bye!
yaaron@GuyHP17:~$ << Guy
> Hey!
> Bye!
> Guy
```

- `Wc`: מדובר בפקודה שעוזרת לניתוח קבצים, אם נרץ סתם `a.txtwc` נקבל כברירת מחדל: כמות שורות, כמות מילים, כמות בתים. ישנם דגלים שמאפשרים לקבל רק נתון ספציפי: `-c` למס' בתים, `-m` למס' תווים, `-w` למס' מילים ו-`-l` למס' שורות (סופר כמה פעמים יש `\n`). נשים לב כי בסיום המס' שורות/תווים/מילים וכו' יופיע גם באותה שורה שם הקובץ.
- הכוח של פקודה זו מגיע משילובה עם פקודות אחרות, למשל `ls |wc` ידפיס כמה קבצים יש בתיקייה הנוכחית.
- `Head`: כברירת מחדל מדפיסה 10 שורות ראשונות של הקובץ/קבצים שצוינו. אם נעשה `num`-יודפסו השורות הראשונות בקובץ.
- `Tail`: כברירת מחדל מדפיסה 10 שורות אחרונות של הקובץ/קבצים שצוינו. אם נעשה `num`-יודפסו השורות האחרונות בקובץ.
- `Pipeline connection`: מקרה של הפניית קלט פלט, פלט של תוכנית אחת הופך לקלט של התוכנית הבאה. למשל `who |wc` יציין את מס' המשתמשים המחוברים. (אין מגבלה על כמה פעמים ששמים |).
- משתנים: **משתנים מקומיים** בשל מוכרים רק לשל, לא לסקריפט/פקודות אחרות. כך נאתחל משתנה: `NEWVAR=" Hey"`

משתני סביבה

כאשר נכנסים shell נקבעת סביבת עבודה, שכוללת את ספריית הבית, סוג הטרמינל עליו עובדים וכו'. ערכים אלו נשמרים בתוך משתני סביבה. משתני סביבה מועברים בין התוכניות והפקודות ולא צריך להגדיר משתנה לפני השימוש בו. בשביל להפוך משתנה רגיל למשתנה סביבה נשתמש בexport.

כדי לשנות ערך של משתנה סביבה, נרשום כך: שם המשתנה בצד שמאל ללא דולר, שווה, לערך החדש בצד ימין. **הערך ישתנה רק עבור הטרמינל הנוכחי והתוכניות שהוא יריץ**, על מנת לשנות ערך משתנה סביבה עבור המערכת כולה יש לערוך קובץ בשם etc/environment. על מנת לראות את כל משתני הסביבה, נוכל להשתמש בפקודה env.

סקריפט (script)

סקריפט הוא תוכנית שמורצת ע"י מפרש (interpreter) ללא קומפילציה. המפענח מפרש את הקוד שורה אחר שורה ומבצע אותה. מפרש נפוץ בהקשר של shells הוא bash. יתרונות – מהירות, יכול עיבוד מחרוזות טובה.

כיצד כותבים סקריפט?

1. נפתח קובץ טקסט, ונוסיף בשורה הראשונה את Shabang: #!/bin/bash (על מנת שהמערכת תדע איזה מפרש יש להריץ).
2. נכתוב את הפקודות שנרצה. (לא צריך סיומת מיוחדת אם כי נהוג ונחמד לשים exit 0).
3. בסוף נריץ.

ארגומנטים לסקריפט: נרצה להעביר לסקריפט ארגומנטים, איך זה עובד? השל לוקח את המילים שכתבנו בשורת הפקודה אחרי שם הסקריפט ומכניס אותן לתוך משתנים מיוחדים - \$0 מכיל את שם הסקריפט עצמו, \$1 את הארגומנט הראשון ששלחנו וכך הלאה... כלומר כך נשלח אותם:

./myscript.sh Guy

איך נוכל להריץ סקריפטים מכל מקום? נרצה להפוך אותו לפקודה שמוכרת ע"י המערכת.

בשלב ראשון: ניצור תיקייה בתוך תיקיית הבית שתשמור את כל הסקריפטים שלנו: mkdir \$HOME/myScripts

בשלב שני: נעביר את הסקריפט שכתבנו לתוך התיקייה הנ"ל: mv dirFiles \$HOME/myScripts

בשלב שלישי: נרצה לעדכן את משתנה הסביבה PATH: PATH=\$HOME/myScripts:\$PATH (מה קורה כאן בעצם? אומרים לשל: כשאתה מחפש פקודות, חפש קודם בתיקיית הסקריפטים שלי ואז בשאר המקומות הרגילים).

כעת, אם נקליד dirFiles מכל מקום בטרמינל, הוא ירוץ כמו פקודה רגילה. **חשוב:** אם נסגור את הטרמינל, השל ישכח את הניתוב הזה.

עוד פקודות וסינטקס:

- Grep: הפקודה מחפשת תבנית טקסט מסוימת בתוך קבצים, ומדפיסה רק את השורות שמכילות תבנית זו. כך סינטקטית: Grep [options] pattern [files]. אפשר להוסיף דגלים: -c למס' השורות שמכילות את התבנית, -i השורות שתואמות לתבנית.

סיכום מערכות הפעלה | © גיא יער-און

- ביטויים אריתמטיים: על מנת שהשל יבין שמדובר בחישוב אריתמטי נשתמש בסוגריים מרובעים. אם נכתוב `num1=$num1+7` התוצאה תהיה כמחרוזת, לכן אם `num1=8` התוצאה תהיה "7+8". לעומת זאת, `num1=${num1+7}` יחשב כמו שצריך.
- הפקודה `test`: הפקודה מחזירה אמת או שקר עבור ביטוי מסוים. כאשר נשים לב כי זה הפוך ממה שאנחנו חושבים לרוב: אמת = 0, שקר = 1.

```
yaaron@GuyHP17:~$ test 1=2
yaaron@GuyHP17:~$ echo $?
0
yaaron@GuyHP17:~$ test 1 = 2
yaaron@GuyHP17:~$ echo $?
1
```

דוגמה: בשורה הראשונה אנחנו מבצעים השמה – ללא הרווח אנחנו פשוט מבצעים השמה, שהצליחה, לכן חוזר 0 ("אמת"), בדוגמה השנייה אנחנו מבצעים בדיקה (יש רווח), לכן חוזר הפעם באמת 1 ("שקר"). כמו כן, `test` זו תוצאת חישוב הטסט האחרון.

- **מערך:** נגדיר מערך באופן הבא – `arr=(x1 x2 x3)` וכו'. נוכל לבצע השמת משתנים באופן הבא: `arr[0]="Hey!"` וכו'. ניגש לאיבר באופן הבא: `echo ${arr[8]}` נגש לכל האיברים בצורה הבאה: `echo ${arr[@]}` או `echo ${arr[*]}`. מה ההבדל? עם `@` ירידת שורה בין כל ערך, ועם `*` בשורה אחת.

Flow Control

נציג כאן סינטקטית מה עלינו לזכור:

```
if condition_command :lf .1
then
  true_commands
else
  false_commands
fi
```

זה הפוך מבדרך כלל, אם `condition command=0` יבוצע `true commends`.

```
while conditions
do
  commands
done :While .2
```

```
for variable in wordlist
do
  commands
done :For .3
```

```
function_name () {
  commands
} :4 פונקציות
```

```
#!/bin/bash

# Define the function
greet() {
  echo "Hello, $1!"
}

# Call the function with an argument
greet "World"
```

לדוגמה:

תרגול 3

- PID: מספר ייחודי שמתקבל בזמן יצירת התהליך.
- UID: מספר ייחודי של המשתמש שמריץ את התהליך.
- PPID: המזהה של התהליך שיצר את התהליך הנוכחי.
- Mode: מצב התהליך (נוצר, ממתין, מחכה וכו').
- Priority: מספר שמסמל את עדיפות הרצת התהליך.

כיצד נוצר תהליך? כאשר מערכת ההפעלה נטענת נוצרים תמיד 2 תהליכים:

1. Pid=0, swapper: אחראי על ניהול הזיכרון.
2. Pid=1, init: כל התהליכים הופכים להיות צאצאים שלו.

Fork()

בלינוקס, על מנת ליצור תהליכים משתמשים בפקודה `fork()`. התהליך המקורי נקרא **תהליך אב**, והתהליך שנוצר נקרא **תהליך בן**. הפעולה משכפלת את תהליך האב לתהליך הבן, וממשיכה לשורת הקוד הבאה בשני התהליכים: התהליך המקורי שקרה לפעולה, והתהליך החדש שנוצר בעקבות הרצת הפעולה. לתהליכים יש:

- קוד זהה (נמצאים באותה נקודה בתוכנית, מיד לאחר הקריאה ל`fork()`!)
- זיכרון זהה (משתנים וערכיהם)
- סביבה זהה (קבצים פתוחים, fd וכו')
- PID שונה!

על מנת להשתמש בפקודה נבצע `#include <unistd.h>`, `#include <sys/types.h>` ונבצע כך: `pid_t fork()`.

ערך מוחזר: במקרה של כישלון, מוחזר -1 לאב. אחרת: לבן מוחזר 0 ולאב מוחזר pid של הבן (`pid > 0`) ולכן זו דרך להבדיל בניהם (בקוד).

- לאחר פעולת `fork()` מוצלחת, לאב ולבן ישנם אותם משתנים בזיכרון אך בעותקים נפרדים. כלומר: שינוי ערכי המשתנים אצל האב לא יראה אותו דבר אצל הבן ולהפך. תהליך הבן הוא תהליך נפרד לחלוטין מתהליך האב!

מנגנון copy on write: ייחודי ללינוקס, מאפשר להשתמש בדפים משותפים לשני התהליכים עד לנקודת הזמן הראשונה בה אחד מן התהליכים מבקש לשנות דף. ואז – מערכת ההפעלה מעתיקה את הדף הזה כך שלכל תהליך יהיה עותק פרטי משלו, ורק אז מאפשרת לתהליך לכתוב על הדף. כלומר: **התהליך נוצר בפועל רק כאשר ישנו שינוי בניהם.**

שאלה. נתבונן בקטע הקוד הבא: מה יודפס?

תשובה. לא ניתן לדעת! ייתכן כי האבא ידפיס קודם, יתכן כי הבן קודם. הדבר היחידי שלא יתכן: 3 יודפס בתחילה. לכן ייתכן:

- 1323 (קודם בן, אח"כ אבא)
- 2313 (קודם אבא, אח"כ בן)
- 2133 (אבא, בן, ואז המשך הקוד)
- 1233 (בן, אבא, ואז המשך הקוד)

```
void main()
{
    pid_t pid;
    if ((pid = fork()) == 0)
        printf("1");
    else
        printf("2");
    printf("3");
}
```


שאלה. נניח ונריץ את קטע הקוד הבא:

Fork()

Fork()

...

Fork()

כמה תהליכים ייווצרו?

תשובה: נסמן את מס' הפעמים שנקראה הפונקציה בח. אזי, כל תהליך פותח 2 חדשים כמעין עץ בינארי מלא. לכן מס' התהליכים כמס' העלים 2^n . אם נרצה לספור את מס' התהליכים החדשים שיווצרו אזי מדובר ב- $2^n - 1$.

- כאשר אב מסיים את פעולתו והבן עדיין חי, הבן נחשב ליתום – orphan. התהליך `init(pid=1)` מאמץ דיפולטיבית את כל היתומים (כלומר עבורם `PPID=1`).

ישנן מספר פונקציות שימושיות בהקשר של `fork()`:

- `Pid_t getpid()`: מחזיר מזהה של תהליך הקורא.
- `Pid_t getppid()`: מחזיר את מזהה תהליך האב של התהליך הקורא.
- `Uid_t getuid()`: מחזיר את מזהה המשתמש של התהליך הקורא.

```
pid_t pid;
pid = getpid();
while (fork() == 0) {
    if (pid == getpid())
        break;
}
```

שאלה. נתבונן בקטע הקוד הבא:

כמה תהליכים ייווצרו בעקבות ביצוע הקוד?

פתרון: ייווצרו אינסוף תהליכים. מדוע? התהליך המקורי קורא ל-`fork()`, יוצר בן ומסיים (כי הערך שחוזר אצלו שונה מאפס). תהליך הבן, נכנס לתוך הלולאה ואצלו לא מתקיים התנאי `pid==getpid()`, כיוון שמזהה תהליך שלו שונה משל האב. לכן נוצר תהליך נכד ותהליך הבן מסתיים (שוב, כי הפעם אצלו `fork()!=0`). למעשה, בכל פעם "יוצרים" ילד חדש, שעבורו לא מתקיים התנאי שבתוך הלולאה ולכן אין ברייק, והתהליך הקודם מסיים את ריצתו ב-`while()` הבא. לכן, נוצר מעין עץ בינארי בצורת "שרוך" אינסופי של היררכיית ילד-אב.

Exec()

נניח כי נרצה להריץ תוכנית מסוימת מתוך תהליך כלשהו, נוכל ליצור תהליך נוסף ע"י פורק, אבל התהליך הזה יהיה זהה לתהליך שממנו נעשתה הקריאה. `Exec` טוענת תוכנית חדשה לביצוע במקום התוכנית שהייתה אמורה להתבצע. להלן הסינטקס:

```
#include <unistd.h>
int execl(char *pathname, char *arg0, ..., NULL)
```

- פרמטר ראשון הוא שם הקובץ שמכיל את התוכנית החדשה לביצוע, החל מן הפרמטר השני: מצביעים למחרוזות שמכילים את הפרמטרים עבור התוכנית הנקראת. באופן מוזר – הפרמטר הראשון חייב להיות שם הקובץ (כך למעשה נעביר לפונקציה פעמיים את אותו איבר), והאחרון חייב להיות `NULL`.

סיכום מערכות הפעלה | © גיא יער-און

- ערך מוחזר: אם יש כישלון יוחזר -1, אחרת: לא נתעניין בערך המוחזר כי הקריאה כלל אינה חוזרת כי exec החליפה את התוכנית הנוכחית בחדשה.

קיימים מספר וריאנטים לפונקציה: `exec{l,v}{optional: e,p}`:

- L: הארגומנטים מועברים כרשימה של מחרוזות המופרדים בפסיקים ומסתיימת בNULL.
- V: הארגומנטים מועברים כמערך של מצביעים `argv*` כאשר האיבר האחרון במערך הוא NULL.
- E: מאפשר להעביר באופן מפורש מערך של משתני סביבה לתהליך החדש.
- P: מדובר במשתנה הסביבה PATH, כך שיעזור לאתר את קובץ ההרצה, ולא נצטרך לתת נתיב מלא של קובץ התוכנית החדשה.

לדוגמה:

```
char *args[] = {"/bin/ls", "-r", "-t", "-l", NULL};  
execv("/bin/ls", args);
```

```
char *args[] = {"ls", "-r", "-t", "-l", NULL};  
execvp("ls", args);
```

:Wait()

כאשר הילד רץ, ההורה: ממשיך לבצע את פעולותיו + ממתין לילד שיסיים. כך נראה סינטקס הפקודה:

```
#include <sys/types.h>  
#include <sys/wait>  
pid_t wait(int *status)
```

- בקריאה לפונקציה, התהליך הקורא (האב) ימתין עד שאחד מבניו יסתיים.
- הערך המוחזר הוא המזהה של התהליך בן שתהליך הקורא ממתין לו.
- הפרמטר status יכיל מספר שניתן יהיה לחלץ ממנו מידע על סטטוס תהליך הבן שהתהליך הקורא ממתין לו.

:Context switch

תהליך יחיד ראשון init נוצר ומקבל PID=1. במהלך ריצתו, נניח והוא יוצר תהליך נוסף שעתיד להתחרות בו על משאבי המערכת. בה"כ PID=2. מערכת ההפעלה תחליף בין PID=1 וכל הנתונים הנלווים לו לבין PID=2 תוך שמירה על חוצץ בין התהליכים. לפני שהתהליך מורדם – נתוניו נשמרים ולפני שהוא "מתעורר" נטענים אותם נתונים לאותם מקומות בדיוק. בשעת ריצת התהליך, הוא רוצה את כל משאבי המערכת הנחוצים לריצתו. אם משאב נחוץ תפוס ע"י תהליך אחר הוא יורדם שוב עד לתור הבא לנסיונו.

:Waitpid()

פונקציה אשר משהה את ביצוע התהליך הנוכחי עד שתהליך בן ספציפי משנה את מצבו – מאפשרת שליטה גמישה על אופן ההמתנה באמצעות options שנבחרו. סינטקטית:

```
pid_t waitpid(pid_t pid, int *status, int options)
```

:exit()

פונקציית ספרייה שקוראת לסיים את התהליך באופן מיידי. Flush הוא הפעולה שבה המחשב מעביר נתונים שנשמרו בזיכרון זמני אל היעד הסופי שלהם (כמו דיסק). ישנן 2 פונקציות רלוונטיות בהקשר זה:

סיכום מערכות הפעלה | © גיא יער-און

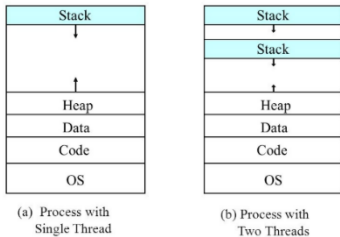
- `Void_exit(int status)`: סיום התהליך קורא באופן מיידי, אין שום חיוב שיהיה `flush`.
 - `Void exit(int status)`: סיום התהליך, כמעט באופן מיידי, כן יתבצע `flush`.
 - `int atexit(void (*function)(void))`: פונקציה זו "רושמת" פונקציה נתונה להיקרא בסיום נורמלי של תהליך או דרך `return` מה `main` של התוכנית. הפונקציות שמוגדרות נקראות בסדר הפוך לסדר הרישום שלהן – כמו מחסנית. אין ארגומנטים שמועברים, אותה פונקציה יכולה להיות רשומה מס' רב של פעמים והיא תקרא פעם אחת פר רישום, מחזירה 0 אם הצליחה, אחרת ערך ששונה מאפס.
- זומבים: כאשר בן מסתיים והאב לא המתין לו, הבן נחשב כזומבי. ה `PCB` שלו עדיין נשמר – כי האבא עדיין יכול לעשות `wait` – רק אז ה `PCB` יימחק. במידה והאב לא יעשה `wait` לבן הבן יהפוך ליתום, והתהליך `init` יאמץ אותו ויעשה לו `wait()`.

תרגול 4

חוסים - Threads

- חוט הוא יחידת ביצוע עצמאית שפועלת בתוך תהליך. (בפועל, מדובר בשורת "עבודה" בתוך תהליך).
- במערכת לינוקס, תהליך אחד יכול לכלול מספר חוסים. חוסים אלו **משתפים ביניהם** את משאבי התהליך! בין היתר data, heap, code וכו' ומשאבים שונים של מערכת ההפעלה.

- עם זאת, למרות השיתוף כל חוט מהווה "הקשר ביצוע" נפרד. לכל חוט יש באופן בלעדי את הדברים הבאים:



1. מחסנית – לניהול קריאות לפונקציות ומשתנים מקומיים.
2. Program counter – מצביע על הפקודה הבאה שהחוט הספציפי אמור לבצע.
3. סט רגיסטרים: המצב הנוכחי של המעבד עבור אותו חוט.

לדוגמה: משמאל תהליך עם חוט אחד, ומימין תהליך עם שני חוסים – ובהתאם זוג מחסניות.

יתרונות של חוסים:

1. ביצוע חלקים ב"ת במשימה: החוסים נועדו לאפשר הרצה בו זמנית של חלקים שונים מתוך משימה של אותו תהליך, שאינם תלויים זה בזה.
2. מקביליות על מעבדים שונים: ניתן להריץ מס' חוסים של אותו תהליך במקביל על גבי מעבדים שונים מה שמזרז משמעותית את זמן הביצוע הכולל.
3. שיפור ביצועים במעבד יחיד: ריבוי חוסים מעיל גם שיש רק מעבד אחד. מדוע? למשל, בזמן שחוט אחד מבצע הוראה חוסמת (למשל: המתנה לקלט/פלט מהדיסק) חוט אחר יכול להמשיך לעבוד ולבצע חלק אחר של התוכנית במקום שהמעבד יתבזבז על המתנה.
- בכל פעם שתהליך נוצר לראשונה נוצר הPrimary thread: חוט יחיד בלבד – החוט הראשוני.
- התקשורת בין החוסים ששייכים לאותו תהליך היא פשוטה ביותר, כיוון שהיא מתבססת על קריאה וכתיבה למשתנים משותפים (שנמצאים באותו מרחב זיכרון).
- עם זאת – זה גם מהווה **חיסרון** משמעותי: יש צורך לתאם את הפעולות בין החוסים שניגשים לאותם משתנים על מנת למנוע מעין שיבוש בנתונים.
- הוספת חוט לתוכנית היא זולה בהרבה מבחינה כלכלית מאשר להוסיף תהליך שלם לאותה המטרה (כלומר, עדיף להוסיף חוסים לתהליך קיים מליצור עוד תהליך). מדוע? הוספת חוט לא דורשת הקצאת משאבים נוספים כמו זיכרון חדש או הרשאות גישה לחומרה, שכן הם משותפים לכל החוסים.

Process Descriptor

- מדובר במבנה שבאמצעותו מערכת ההפעלה מנהלת את כל התהליכים שקיימים.
- במערכת לינוקס, לכל תהליך קיים בגרעין (Kernel) מבנה נתונים שנקרא מתאר תהליך – process descriptor: מדובר ברשומה שמרכזת את כל המידע הרלוונטי עבור אותו תהליך. מכיל את השדות הבאים:

1. מצב התהליך: אם רץ, ממתין או סיים.
2. עדיפות התהליך: כמה זמן מעבד התהליך יקבל ביחס לתהליכים אחרים.
3. מזהה התהליך – PID

4. מצביע לטבלת אזורי הזיכרון: מידע על ניהול הזיכרון של התהליך.
 5. מצביע לטבלת הקבצים הפתוחים – רשימת כל הקבצים שהתהליך משתמש בהם כרגע.
- וכו'. Kernel משתמש במידע הזה בשביל לדעת איך ומתי להריץ כל תהליך.

POSIX Threads:

כעת נדון כיצד מממשים חוטים במובן המעשי בלינוקס.

התמיכה בחוטים בלינוקס שואפת להתאים לתקן הכללי למימוש חוטים במערכת UNIX שנקרא POSIX Threads. לפי תקן זה, כל החוטים של אותו תהליך צריכים להיות מאוגדים יחד. המשמעות היא: קריאה לפונקציה `getpid()` מכל חוט של אותו תהליך אמורה להחזיר את אותו ערך (מזהה התהליך המשותף).

בלינוקס, יש קאץ':

- בלינוקס, היחס לכל חוט הוא כאל תהליך רגיל ולכן לכל חוט ישנו מתאר משלו ובפרט PID משלו.
 - אך, ישנה סתירה – כי המתכנת בהתאם לתקן POSIX מצפה שכל החוטים ששייכים לאותו תהליך יזוהו דרך PID יחיד.
- לכן, המערכת מצפה שפעולות שמתבצעות על ה-PID של התהליך ישפיעו על כל החוטים בתוך אותו תהליך, ופעולת `getpid()` בכל חוט תחזיר את אותו PID באותו תהליך. לכן, לינוקס משתמש בקבוצת חוטים:
- Thread group:** איחוד חוטים – כל החוטים ששייכים לאותו תהליך נמצאים יחד תחת קבוצה אחת. קבוצת החוטים היא יישות שמוגדרת בקרנל של לינוקס, הקבוצה נועדה לאפשר התייחסות משותפת לכל החוטים באותו תהליך.
- לכן, על אף שלינוקס יודעת להבדיל בחוטים בפנים – כלפי חוץ יש לכל החוטים אותו PID.
- כיצד לינוקס מממשת את הקבוצה הנ"ל? על מנת לאחד את כל החוטים תחת תהליך אחד, לינוקס משתמשת בשדות הבאים:
- `Thread_group`: מתארי התהליך של כל החוטים באותה קבוצה מקושרים פיזית באמצעות השדה למתאר התהליך של `primary thread` (כל החוטים מחוברים לחוט הראשון באותו תהליך).
 - `Tgid (thread group id)`: זהו ה"שקר המוסכם" של לינוקס, השדה הזה מכיל את ה-PID המשותף לכל החוטים באותה קבוצה. בפועל הוא יהיה ערך ה-PID של החוט הראשון באותו תהליך.

נתבונן בדוגמה:

Thread A	Thread B	Thread C
PPID = 2900	PPID = 2900	PPID = 2900
TGID = 3000	TGID = 3000	TGID = 3000
TID = 3000	TID = 3001	TID = 3002

נניח כי התהליך עם PID=3000 שמכיל 3 חוטים.

התהליך שיצר את התהליך הנוכחי – PPID מזהה ע"י תהליך 2900. הגיוני שלשלושתם אותו ערך שכן החוטים נמצאים באותו תהליך.

ה-TGID – ה-PID המשותף: שהוא ערך ה-TID הראשון.

- לכן, בלינוקס קריאה ל-`getpid()` תחזיר את ה-TGID של החוט הקורא (מזהה התהליך). פעולות על ה-PID המשותף מתורגמות לפעולה על קבוצת החוטים המתאימה ל-PID.

כיצד זה נראה בקוד?

ראשית, יש לבצע `include` לספרייה `<pthread.h>`.

סיכום מערכות הפעלה | © גיא יער-און

```
int pthread_create(  
    pthread_t *thread,  
    pthread_attr_t *attr,  
    void *(*start_func)(void*),  
    void *arg);
```

- יצירת חוט: קוראים לפונקציה pthread_create. מה הפונקציה הנ"ל עושה? היא יוצרת חוט חדש שמתבצע במקביל לחוט הקורא שקרא לה, ושניהם רצים יחד בתוך אותו תהליך. אם היצירה הצליחה – החוט החדש יתחיל להריץ את הפונקציה ששלחנו לו בפרמטר start_func. החוט ייהרג אוטומטית בעת שיסיים לבצע את הפונקציה הנ"ל. אלו הפרמטרים של הפונקציה:

1. Thread: מצביע למשתנה שבו המערכת תשמור את המזהה של החוט החדש.
 2. Attr: מאפיינים מיוחדים לחוט למשל אם מדובר בחוט מערכת או משתמש (שולחים לרוב NULL כברירת מחדל)
 3. Start_func: שם הפונקציה שהחוט החדש צריך להריץ. הערך המוחזר מפונקציה זו במקרה של סיומה הטבעי הוא ערך הסיום של החוט.
 4. Arg: הארגומנט הנתון שאנחנו רוצים להעביר לפונקציה שהחוט מריץ.
- מדובר בפונקציה אסינכרונית – בעת שקראנו לה, החוט החדש יוצא לדרך והחוט המקורי ממשיך לשורה הבאה מבלי לחכות לו. הפונקציה מחזירה 0 אם הצליחה, והמזהה של החוט נשמר בthread. או – קוד שגיאה (ערך חיובי ממש) במקרה של כישלון. למעשה, מדובר בפונקציה דומה לfork() רק שfork() משכפלת תהליך – וכאן מוסיפים "פועל" נוסף ידנית לתהליך.

```
#include <pthread.h>
```

```
pthread_t pthread_self();
```

- Pthread_self: לפעמים חוט צריך לדעת את המזהה שלו – למשל על מנת להשוות את עצמו למזהה אחר וכו'. החוט שקורא לפונקציה מקבל את המזהה של עצמו. המזהה הנ"ל הוא מזהה פנימי לספרייה pthread ואינו קשור ישירות לpid\tid עליהם דיברנו קודם – מדובר במזהה שקבעה הספרייה על מנת לנהל את החוטים בקוד שלנו. ערך מוחזר הוא משתנה מהצורה pthread_t שמייצג את מזהה החוט.

```
#include <pthread.h>
```

```
int pthread_join(pthread_t th, void **thread_return);
```

- Pthread_join: בדומה לפונקציה wait() של תהליכים שמאפשרת להורה לחכות לבנו. כאשר חוט אחד יוצר חוט אחר הוא לרוב צריך לחכות לו על מנת לקבל ממנו תוצאות או פשוט בכדי לוודא שהוא סיים לפני שהתוכנית כולה נסגרת. החוט שקורא לפונקציה עוצר את הביצוע שלו וממתין עד שהחוט בעל המזהה th יסיים את עבודתו. ההמתנה הנ"ל קריטית כי היא משחררת את כל מידע הניהול של החוט בתוך הספרייה והקרנל, בלי זה המערכת עלולה להשאר עם שאריות של חוטים מתים.

נבחין:

1. ניתן להמתין לסיום של חוט מסוים רק פעם אחת! אם ננסה לקרוא לפונקציה על אותו חוט פעמיים מדובר בהתנהגות שאינה מוגדרת.
 2. כל חוט בתהליך יכול להמתין לכל חוט אחר באותו תהליך (לא רק כמו "אבא שממתין לילד").
 3. אם כמה חוטים שונים מנסים לחכות לחוט אחד בו זמנית – התוצאה לא מוגדרת ותלויה בתזמון הריצה.
- Pthread_cancel: לפעמים נרצה להפסיק חוט באמצע עבודתו. לפעמים גם – המשימה שהחוט מבצע כבר הופכת ללא רלוונטית (למשל: המשתמש לחץ "בטל"). הפונקציה שולחת בקשה לסיים את הביצוע של החוט שמזוהה ע"י הפרמטר thread. זוהי בקשה – ולא פקודה חד משמעית והוראה. החוט לא תמיד מת מיד, זה תלוי בנקודת הביטול שלו ובמצב הביטול שהוגדר לחוט (מוכן / לא מוכן לקבל ביטול – מגדירים זאת בקוד עצמו לרוב). אם מוחזר 0 – הבקשה נשלחה בהצלחה, אם הפעולה נכשלה (חוט לא קיים) יוחזר מס' חיובי.
 - Pthread_exit: בניגוד לcancel, כאן החוט מחליט על דעת עצמו לסיים את תפקידו. החוט שקורא לפונקציה מסיים את פעולתו באופן מיד. הפרמטר retval מצביע לערך שהחוט רוצה להחזיר – מי שיקבל את הערך הזה הוא החוט שיבצע join (חיכה שהחוט הנ"ל יסיים) על החוט הנ"ל (דומה לקוד חזרה של תהליך). אם החוט הראשי (main) קורא

סיכום מערכות הפעלה | © גיא יער-און

לפונקציה הוא מסיים את עצמו אך לא סוגר את התהליך, שאר החוטים ימשיכו לרוץ עד שהאחרון בהם יסיים. זה שונה מexit/return וכו'.

- סיום של תהליך: אם חוט כלשהו מתוך תהליך קורא לexit() מתבצע סיום ביצוע התהליך כולו. כל החוטים בקבוצה של התהליך מפסיקים. לאחר סיום ביצוע קוד החוט הראשי מתבצעת קריאה אוטומטית לexit() שגורמת לסיום ביצוע התהליך. אם כל החוטים בתהליך מסיימים באמצעות pthread_exit() אזי סיום החוט האחרון הוא השלב שבו מסתיים התהליך.
- סיום של חוט: חוט יכול להסתיים באחת מן הסיבות הבאות –
 1. חזרה מהפונקציה הראשית של החוט
 2. קריאה לpthread_exit() בתוך החוט
 3. קריאה לexit() ע"י חוט כלשהו בקבוצה של החוט המדובר.
 4. סיום טבעי של החוט הראשי.
 5. הריגת החוט ע"י קריאה לpthread_cancel() מחוט אחר בתהליך (או מהחוט עצמו).

קריאה לfork() בתוך חוט

כאשר חוט קורא לfork() נוצר תהליך חדש שהוא הבן של החוט הקורא בלבד. חוט אחר בקבוצה של החוט הקורא יכול לבצע wait() על תהליך הבן שנוצר. כלומר – למרות שחוט ספציפי יצר את התהליך שנוצר הופך להיות תהליך בן של התהליך כולו. לתהליך הבן החדש יכולים להיות חוטים משלו – בהתחלה כמובן שחוט יחיד. אך חוטים נוספים יכולים להתווסף בהמשך תהליך הבן. נבהיר: גם אם לתהליך המקורי היו עוד חוטים, לתהליך הבן משתכפל רק חוט אחד. במידה ותהליך הבן יכיל יותר מחוט אחד, חוט האב יכול לבצע wait() על תהליך הבן פעם אחת בלבד, להמתין לסיום תהליך הבן.

קריאה לexec() בתוך חוט

כשאחד החוטים בתהליך קורא לexec() קורה תהליך דרסטי: ברגע שהקריאה מצליחה, המערכת עוצרת ומוחקת את כל שאר החוטים שהיו קיימים בתהליך! הם לא מקבלים הודעה מראש ולא מחכים להם עם join: אלא פשוט מפסיקים להתקיים באותה שניה. החוט הספציפי שביצע את הקריאה הוא הניצול היחיד – והופך להיות החוט הראשי של התוכנית החדשה שנטענה. כיוון שexec() טוענת קובץ הרצה חדש, כל מרחב הכתובות של התהליך נמחק ונוצר מחדש. זה כולל את הזיכרון, קוד התוכנית ומשתנים גלובליים. הכל מתחיל מחדש עבור התוכנית החדשה. נבחין שזה הגיוני – קריאה לפונקציה זו אמורה להחליף את הקוד שבוצע, ולא הגיוני שחוטים אחרים יוכלו להמשיך במקביל.

Mutex

מדובר במנגנון סנכרון שנועד למנוע מכמה חוטים לגשת למשאב משותף בו זמנית ולגרום לשיבוש בנתונים. הוא עובד כמו מפתח יחיד לחדר, מי שמחזיק במפתח נכנס פנימה וכל השאר חייבים לחכות בחוץ עד שהוא יצא ויחזיר את המפתח.

- מדובר במשתנה פשוט שיכול להיות או נעול או לא נעול.
- משתמשים בו בכדי להקיף קוד רגיש שבו ניגשים לנתונים משותפים. נועלים בכניסה ומשחררים ביציאה כדי להבטיח שרק חוט אחד נמצא שם בכל רגע נתון.
- המנעול מאפשר לכל היותר לחוט אחד להחזיק בו.

סיכום מערכות הפעלה | © גיא יער-און

- המתנה – אם חוט מנסה לנעול מיוטקס שכבר תפוס על ידי חוט אחר הוא נחסם עד שהמנעול מתפנה.
- למיוטקס יש "בעלים" – רק החוט שנעל את המיוטקס הוא הבעלים שלו ורק הוא מורשה לשחרר אותו. חוט אחר לא יכול לבוא ולפתוח את המנעול שמישהו אחר שם.
- ולסיכום: מיוטקס הוא מנגנון הגנה שמאפשר רק לחוט אחד בכל פעם לגשת לקוד רגיש, כדי למנוע שיבוש נתונים שנגרם מגישה מקבילית. החוק המרכזי הוא שרק החוט שנעל את המיוטקס רשום כבעליו והוא היחיד שמורשה לשחרר אותו, בזמן שכל חוט אחר שינסה לנעול אותו ייחסם וימתין לשחרורו.

```
#include <pthread.h>
int pthread_mutex_init(
    pthread_mutex_t *mutex,
    const pthread_mutexattr_t *attr);
```

- אתחול mutex: כדי שהמערכת תדע לנהל את המנעול, חייבים לאתחול באמצעות הפונקציה.
- 1. Mutex: מצביע (כתובת) למשתנה מסוג pthread_mutex_t שהגדרנו, זה האובייקט שייצג את המנעול בפועל.
- 2. Attr: מאפייני המנעול. כאן קובעים את סוג המיוטקס:
- PTHREAD_MUTEX_NORMAL: מיוטקס רגיל ומהיר – הוא לא בודק טעויות (כמו נסיון של חוט לנעול מנעול שהוא כבר נעל את עצמו).
- PTHREAD_MUTEX_RECURSIVE: מיוטקס רקורסיבי. מאפשר לאותו חוט לנעול את המנעול כמה פעמים ברצף בלי להחסם.
- PTHREAD_MUTEX_ERRORCHECK: מיוטקס שבודק שגיאות ומחזיר קוד שגיאה אם חוט מבצע פעולה לא חוקית (שחרור מנעול של חוט אחר).
- PTHRED_MUTEX_DEFAULT: שימוש ברירת המחדל של המערכת.

נבחין – אסור לאתחל מיוטקס שכבר אותחל בעבר. נסיון כזה יוביל להתנהגות לא מוגדרת.

ערך מוחזר: 0 אם הצליח, מס' חיובי אם לא.

סוג ה-mutex	נעילה חוזרת ע"י החוט המחזיק במנעול	שחרור מנעול ע"י חוט שאינו מחזיק במנעול	שחרור מנעול שאינו נעול
PTHREAD_MUTEX_NORMAL	DEADLOCK	תוצאה לא מוגדרת	תוצאה לא מוגדרת
PTHREAD_MUTEX_RECURSIVE	הצלחה, מגדיל מונה נעילה עצמית ב-1	הפעולה נכשלת	הפעולה נכשלת
PTHREAD_MUTEX_ERRORCHECK	הפעולה נכשלת	הפעולה נכשלת	הפעולה נכשלת
PTHREAD_MUTEX_DEFAULT	ב-Linux, מתנהג כמו PTHREAD_MUTEX_NORMAL.		

ניתן לבצע מס' פעולות:

- נעילת mutex:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

הפעולה חוסמת עד שה-mutex מתפנה ואז נועלת אותו.
- נסיון לנעילת mutex:

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

הפעולה נכשלת אם ה-mutex כבר נעול, אחרת נועלת אותו.
- שחרור mutex נעול:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

ניתן גם להרוס mutex:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

הריסת מנעול גורמת לכך שלא יהיה אפשר להשתמש בו. כדי להשתמש שוב במנעול אפשר להפעיל עליו את הפונקציה pthread_mutex_init. הריסת מנעול שנמצא במצב נעול או לא מאותחל תגרור תופעה לא מוגדרת.

תרגול 5

```
#include <sys/stat.h>
#include <fcntl.h>
int open(const char *pathname, int flags);
```

קריאת המערכת `Open()`:

קריאת המערכת משמשת לפתיחת קבצים. הפרמטר `flags` קובע את אופן פתיחת הקובץ:

- `O_RDONLY`: פתיחת הקובץ לקריאה בלבד.
- `O_WRONLY`: פתיחת הקובץ לכתיבה בלבד.
- `O_RDWR`: פתיחת הקובץ לקריאה ולכתיבה.
- `O_CREATE`: פתיחת הקובץ ויצירתו במידה ואינו קיים.
- `O_EXCL`: גורם לפקודת `open` להכשל אם הקובץ כבר קיים.
- `O_TRUNC`: פתיחת הקובץ וקיצוץ גודלו לאפס.
- `O_APPEND`: הכתיבה לקובץ תהיה לסופו ולא נדרוס מידע קיים.

אם הפונקציה מצליחה הערך החוזר הוא `fd`, ואחרת -1.

ניהול תיקיות:

כעת נעבוד לדון מרמת הקובץ הבודד לניהול של תיקיה. המטרה היא לפתוח סטרים שמאפשר לקרוא תוכן של תיקיה שלמה.

```
#include <dirent.h>
DIR* opendir(const char* pathname);
```

על מנת לגשת למידע באשר לתיקיה נשתמש בפונקציה:

אם הפונקציה הצליחה – יוחזר מצביע למבנה מסוג `DIR`. המצביע הוא `stream` של מידע, שבעת היצירה ממוקם ברשומה הראשונה בתיקיה ומכיל את המיקום הנוכחי בה (כלומר לא מחזירים תיקיה, אלא מקבלים מצביע למבנה נתונים שמייצג את התיקיה ובעת שהוא נוצר הסמן שמוחזר הוא העמודה הראשונה בתוך התיקיה). בכישלון יוחזר `NULL`.

```
int closedir(DIR* dp);
```

על מנת לסגור משתנה מסוג `DIR` המייצג ספרייה פתוחה – נשתמש בפונקציה:

```
struct dirent {
    ino_t d_fileno; // i-node number.
    char d_name[NAME_MAX + 1]; // file name
    ...
}
struct dirent* readdir(DIR* dp);
```

כעת לאחר שפתחנו את התיקיה, נרצה לקרוא את התוכן שבה קובץ אחר קובץ. הפונקציה `readdir()` היא הדרך שלנו לעבור על רשימת הקבצים שבתוך התיקיה. כל קריאה אליה מחזירה לנו מידע על רשומה אחת (קובץ או תיקיית משנה) שנמצאת שם.

`Dirnet` הוא מבנה הנתונים שמכיל את המידע על הקובץ הספציפי שקראנו ומכיל שני שדות עיקריים:

1. `D_fileno` מס' `i_noden` של הקובץ במערכת.

2. `D_name`: מחרוזת שמכילה את שם הקובץ.

כיצד משתמשים בפונקציית הקריאה? כדי לקבל מידע על רשומה מזמנים אותה. כל קריאה לפונקציה תחזיר מידע על הרשומה הבאה בתיקיה – כמו איטרטור (הקריאה הראשונה תחזיר את הקובץ הראשון, השנייה את השני וכו'). כשנגמרים הקבצים בתיקיה הפונקציה תחזיר `NULL`.

דוגמה לסריקה של תיקיה (משמאל):

```
#include <stdio.h>
#include <dirent.h>

main(int argc, char **argv)
{
    DIR *pDir;
    struct dirent *pDirent;
    if ((pDir = opendir(argv[1])) == NULL)
        exit(1);
    // looping through the directory, printing the directory entry
    name
    while ((pDirent = readdir(pDir)) != NULL)
        printf("%s\n", pDirent->d_name);
    closedir(pDir);
}
```

:Read syscall

```
int read(int fd, void* buff, size_t nbytes);
```

על מנת לקרוא מנתונים מקובץ נשאוב ממנו נתונים לתוך משתנה בזיכרון של התוכנית באמצעות הפונקציה. הbuff הוא המצביע למיקום בזיכרון אליו יישפכו הנתונים שנקראו. כמו כן נעביר את כמות הבתים שנרצה לקרוא מקובץ ואת fd. הקריאה מתבצעת עד שהגענו למס' הבתים שנדרשו / נגמר המיקום בקובץ.

:Write syscall

```
int write(int fd, void* buff, size_t nbytes);
```

המטרה בפונקציה זו היא להעביר נתונים מזיכרון התוכנית buff לתוך קובץ המזוהה ע"י fd. אם הוחזר 1- הפונקציה נכשלה ואחרת יוחזר מס' הבתים שנכתבו לתוך הקובץ.

:Close syscall

```
int close(int fd);
```

באופן עקרוני, כל קובץ שנשאב פתוח כאשר התהליך שפתח אותו ייסגר ע"י מערכת ההפעלה. סגירת fd זה בעצם עדכון open file descriptor שיש לו פחות מצביע אחד. כאשר המונה של מס' המצביעים לfd מגיע לאפס אזי הוא (ofd) משתחרר גם כן.

```
main(int argc, char argv)
{
    int fdin; /* input file descriptor */
    int fdout; /* output file descriptor */
    char buf[SIZE+1]; /* input (output) buffer */
    int charsr; /* how many chars were actually read */
    int charsw; /* how many chars were actually written */

    fdin = open("dugma1.txt", O_RDONLY);
    if (fdin < 0) /* means file open did not take place */
    {
        perror("after open"); /* text explaining why */
        exit(-1);
    }

    /* create the file with read-write permissions */
    fdout = open("dugma2.txt", O_CREAT | O_RDWR, 0666);
    if (fdout < 0) /* means file open did not take place */
    {
        perror("after create"); /* text explaining why */
        exit(-1);
    }

    do
    {
        charsr = read(fdin, buf, SIZE);
        /* why writing SIZE can be wrong... */
        charsw = write(fdout, buf, charsr);
        if (charsw < charsr)
        {
            printf("not all bytes were written.\n");
        }
    } while ((charsr == SIZE) && (charsw == SIZE));

    close(fdin); /* free allocated structures */
    close(fdout); /* free allocated structures */
}
```

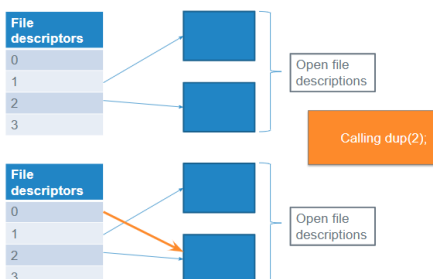
דוגמה לקוד פשוט שמעביר תוכן מקובץ אחד לאחר:

```
#include <unistd.h>
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

משפחת dup:

קריאות המערכת הנ"ל מאפשרות לשכפל fd, מה שיאפשר לכמה מספרים בטבלת fd להצביע על אותו קובץ פתוח. למה שנרצה לעשות זאת בכלל? עבור ניתוב מחדש – בלינוקס הרי יש שלושה ערוצים שתמיד פתוחים. נניח וכתבנו תוכנית עם הרבה פקודות הדפסה, במקום לשנות את כל הקוד כך שייכתב לקובץ, נוכל להשתמש בdup2 על מנת לומר למערכת ההפעלה "מעכשיו כל מה שאמור ללכת למסך, תשלחי אותו לfd של הקובץ שפתחתי".

הפונקציה הראשונה יוצרת עותק של oldfd, המערכת מחפשת בטבלה את האינדקס הפנוי הנמוך ביותר ומעתיקה אליו את ההצבעה של oldfd. כתוצאה שני fds מצביעים לאותו הקובץ. הפונקציה השנייה, מאלצת את newfd להצביע לאן שoldfd מצביע.



I/O Redirection

מה זה? היכולת להפנות את הקלט או הפלט הסטנדרטיים של התוכנית שבדרך כלל מחוברים למקלדת, למקומות אחרים. בטרמינל של של, אנחנו משתמשים בסימנים מיוחדים לבצע זאת. הסימון < גורם לתוכנית לכתוב את הפלט לקובץ במקום למסך. למשל: `ls > file.txt` ינתב את ההדפסה לתוך הקובץ. בפועל, מאחורי הקלעים זה ממומש באמצעות `dup`: בשלב הראשון פותחים את הקובץ המבוקש, ומקבלים `fd`. בשלב השני משתמשים ב`dup2` על מנת להעתיק את `fd` של הקובץ לתוך `fd` של `IO`.

לדוגמה: בקוד הבא, פעולת ההדפסה האחרונה כבר לא תגיע למסך (1) אלא לקובץ:

```
}
printf("This goes to the standard output.\n");
printf("Now the standard output will go to '%s'\n", argv[1]);
dup2(newfd, 1);
printf("This goes to the file.\n");
exit(0);
```

עד כה קראנו מתחילת/אמצע קובץ. כעת ניתן יהיה לגשת לקרוא/לכתוב מכל מקום שהוא בקובץ. קריאת המערכת `lseek` מאפשרת לשנות את ה`offset` הנוכחי של `FD`. ה`whence` יהיה אחד מהבאים:

- `SEEK_SET`: ההיסט מחושב מתחילת הקובץ.
- `SEEK_CUR`: ההיסט מחושב מהמיקום הנוכחי של הסמן.
- `SEEK_END`: ההיסט מחושב מסוף הקובץ.

במקרה של הצלחה הפונקציה מחזירה את מיקום הסמן החדש. אחרת, -1.

למשל על מנת לדעת את גודלו של קובץ נוכל לבצע: `lseek(fd, 0, SEEK_END)`

Chmod

```
#include <sys/types.h>
#include <sys/stat.h>
int chmod(const char* path, mode_t mod);
```

הפונקציה הנ"ל אחראית על אבטחה והרשאות במערכת הקבצים. היא מאפשרת לשנות למי מותר לקרוא, לכתוב או להריץ קובץ מסוים. ה`path` הוא הנתב לקובץ שאת הרשאותיו נרצה לשנות. ה`mod` הוא ערך חדש של הרשאות (מיוצג לרוב כערך אוקטלי או כשילוב ביטים).

:Errno

```
#include <errno.h>
extern int errno;
```

כמעט כל קריאת מערכת בלינוקס מעדכנת משתנה גלובלי בשם `errno` במקרה של שגיאה. המשתנה מכיל קוד מספרי שמייצג את סיבת השגיאה הספציפית שקרתה.

לכל קריאת מערכת יכולות להיות סיבות אחרות לכישלון – קובץ לא קיים, אין הרשאות וכו'. בעת שגיאה המתכנת צריך לבדוק את ערך המשתנה הגלובלי ולפי המס' הזה להדפיס הודעה מתאימה למשתמש.

```
#include <stdio.h>
void perror(const char *s);
```

`Perror`: הפונקציה מדפיסה הודעת שגיאה ל `stderr` אשר מתארת את השגיאה האחרונה שהתרחשה במהלך קריאה ל `system call` או פונקציית ספריה. יודפס ה-`string` שהועבר, לאחר מכן נקודתיים ורווח, והודעת השגיאה המתאימה ל-`errno`.

כעת נשאל את עצמנו שאלה.

שאלה. האם כל קריאה ל`read` מבצעת פיזית קריאה מהדיסק?

סיכום מערכות הפעלה | © גיא יער-און

תשובה. לא! מערכת ההפעלה מנסה להימנע מכך ככל האפשר כי קריאה מהדיסק יקרה מאוד. לכן נוקטים באסטרטגיית הבלוקים: כאשר התוכנית מבקשת לקרוא בייט בודד, מערכת ההפעלה לא ניגשת לדיסק בשביל בייט בודד אלא קוראת בלוק שלם של נתונים ושומרת אותו בקאש. אם נבקש לקרוא את הבייט הבא, המערכת תביא אותו הישר מהזיכרון המהיר מבלי לגשת לדיסק שוב.

ישנן פונקציות מספריית f, כמו fread and fwrite. בניגוד לwrite שקוראת באופן ישיר למערכת ההפעלה, פונקציות אלו מנהלות חוצץ פנימי בזיכרון התוכנית עצמה: כאשר קוראים לפונקציות אלו, המידע לא נשלח מיד לדיסק אלא נשמר בבאפר בuser-space. רק כאשר הבאפר מתמלא, הפונקציה מבצעת קריאת מערכת אחת גדולה לwrite/read. זה חוסך overhead של מעבר בין יוזר לקרנל מוד. מה הבעיה? החסכון בביצועים מגיע עם מחיר של חוסר ודאות מסוים – אם התהליך קורס או מסתיים בפתאומיות מידע שנשאר בבאפר לא ישפך לדיסק אלא יעלם. וכן, המידע יכול להיות תקוע בזיכרון מבלי שנכתב באמת. **למעשה: הרעיון של פונקציות אלו הוא "לשמור" מידע עד שנשמר מספיק מידע – ואז לבצע פקודת מערכת.**

על מנת לפתור את הבעיה הזו – נרצה לפעמים לחייב כתיבה פיזית. יש 2 דרכים לעשות זאת:

- שימוש בדגל O_SYNC בעת פתיחת קובץ. זה מודיע למערכת ההפעלה שכל פעולת כתיבה לקובץ זה חייבת להיות מסונכרנת. כלומר כל קריאה תהיה חסומה – התוכנית תמתין לקריאת המערכת ולא תתקדם לשורה הבאה עד שמערכת ההפעלה תאשר שהמידע נכתב פיזית על הדיסק.
- שימוש בפונקציה fsync: לפעמים לא נרצה להאט את כל הכתיבות, אלא רק לוודא בנק' זמן מסוימות שהכל נשמר. פעולה זו מכריחה את מערכת ההפעלה לרוקן את כל הבאפר שנמצא בזיכרון ישירות לדיסק באותו רגע. נשתמש בה לאחר סדרת כתיבות לבאפר.

```
#include <stdio.h>
int fprintf(FILE *restrict stream, const char *restrict format, ...);
```

```
#include <stdio.h>
int fscanf(FILE *restrict stream, const char *restrict format, ...);
```

- הפונקציה fprintf: מאפשרת הדפסה בפורמט printf לתוך קובץ.
- הפונקציה fscanf: מאפשרת קריאה בפורמט scanf מקובץ.

תרגול 6

Critical Section: מדובר בקטע בקוד שבו התהליך ניגש למשאב משותף (כמו משתנה גלובלי, קובץ או זיכרון משותף). אם שני התהליכים ינסו לשנות את אותו משתנה בקטע זמן זה – הנתונים ישתבשו. Semaphore הוא מנגנון שמוודא שרק תהליך אחד בכל פעם נמצא בקטע הקריטי.

Semaphore הוא מנגנון שמגן על קטע קוד זה.

- הסמפור הוא אובייקט (משתנה) שכל התהליכים רואים. הוא משמש כנקודת התיאום בניהם.
- בתוך הסמפור יש מונה (counter). כשתהליך רוצה להכנס לקטע הקריטי, הוא חייב להוריד באחד את ערך המונה. אם המונה הוא אפס התהליך נאלץ לחכות.
- Atomic operations: הבדיקה של המונה וההורדה שלו באחד קורות בפעולה אחת בלתי ניתנת לחלוקה – אי אפשר ששני תהליכים יבדקו את המונה ויורידו בו זמנית ויחשבו ששניהם יכולים להכנס.

ישנם שני סוגי סמפורים:

1. בינארי: הערך שלו יכול להיות רק 1 או 0. הוא מתנהג כמו מנעול – או שהוא פתוח או שהוא סגור. מאפשר רק לתהליך אחד בדיוק להכנס לקטע הקריטי בכל פעם.
2. Counting semaphore: הערך שלו יכול להיות כל מספר, הוא מאפשר למספר מוגדר של תהליכים להכנס לקטע הקריטי בו זמנית.

כיצד סמפור פועל?

- הגדרת critical section: מגדירים איזה חלק בקוד הוא המסוכן.
- יצירת סמפור – יוצרים אובייקט סמפור שיגן על קטע הקוד הנ"ל.
- הכניסה (הנעילה): הראשון זוכה – התהליך הראשון שמגיע לקטע הקוד הנ"ל נועל את הסמפור ונכנס פנימה. אם מגיעים תהליכים נוספים בזמן שהראשון בפנים, הם לא סתם נעלמים. הסמפור מנהל רשימת ממתינים – תהליכים אלו נכנסים למצב המתנה, ולא מבזבזים זמן מעבד.
- היציאה: כשתהליך ראשון מסיים את העבודה שלו בתוך הקוד הקריטי – הוא יוצא ומאזנת לסמפור שסיים. הסמפור מסתכל ברשימת הממתינים, מעיר את אחד התהליכים שחיכו ונותן לו את רשימת הכניסה לקטע הקוד.

בפועל – מערכת ההפעלה דואגת לביצוע כל פעולות ההמתנה והאיתות.

ישנם שני סוגי סמפורים בלינוקס, אנחנו נתמקד באחד בשם: **System V Semaphores**.

```
#include <sys/sem.h>
```

```
int semget(key_t key, int nsems, int semflg)
```

1. Semget: ראשית נבחין כי בלינוקס אנחנו לא יוצרים סמפור בודד אלא מערך של סמפורים. היא יוצרת סמפור (או מערך סמפורים) ארגומנטים שלה הם:

- Key_t key – שם הסמפור במערכת ההפעלה. אם תהליכים שונים רוצים להשתמש באותו הסמפור, הם ישתמשו באותו מפתח. IPC_PRIVATE הוא ערך מיוחד שאומר "הסמפור הנ"ל הוא עבורי ועבור ילדיי" (תהליך אומר זאת). תהליכים זרים לא יוכלו לגשת אליו.
- Int nsems: כמה סמפורים נרצה בתוך הסט (מערך) הנ"ל. בגישה לסמפור קיים – אפס. ביצירת חדש – ערך גדול מאפס (לרוב 1).

סיכום מערכות הפעלה | © גיא יער-און

- `int semflg`: קובעים איך הפונקציה תתקיים. ישנם מס' דגלים: `IPC-EXCL` אצי אם הסמפור כבר קיים הפונקציה תכשל.
 הערך שהפונקציה מחזירה הוא מס' שלם, בהצלחה מחזירה מזהה ייחודי לסמפור (מזהה שלו). בכישלון -1.

```
#include <sys/sem.h>
int semop(int semid, struct sembuf *sops, unsigned nsops)
```

2. **Semop**: זו הפונקציה שמבצעת את פעולות הנעילה והשחרור בפועל.

`semid` הוא הערך שקיבלת כתוצאה מהפונקציה הקודמת. זה מזהה הסמפור.

- **Sops**: מצביע למערך של מבנים מסוג `sembuf` – כל איבר במערך מתאר פעולה על סמפור אחד בתוך הסט.
- **Nsops**: כמה פעולות יש במערך הנ"ל (לרוב יהיה 1).

בתוך הסטרוקט הנ"ל, ישנם 3 שדות שקובעים מה יקרה.

- `Sem_num`: על איזה סמפור בתוך הסט אנחנו פועלים (0 לראשון).
- `Sem_op`: סוג הפעולה. חיובי (1) זה שחרור משאב – מוסיף ערך למספר הסמפור ומעיר תהליכים שמחכים. שלילי (-1) זה נעילת משאב – אם ערך הסמפור גדול מהערך המוחלט אנחנו מחסירים אותו וממשיכים. אם הערך הוא אפס, התהליך נעצר ומחכה עד שהסמפור יעלה. אם הוא אפס – ממתינים עד שהסמפור יהיה בדיוק אפס (ניצול מלא של המשאב).
- `Sem_flg`: ישנו דגל חשוב מאוד `SEM_UNDO` (יחד עם עוד דגלים שקיימים וכו'), דגל זה אומר למערכת ההפעלה – אם התהליך קורס בעת שהוא מחזיק בסמפור, תחזירי את הסמפור למצבו הקודם בצורה אוטומטית. מונע מצב בו שאר התהליכים יחכו לנצח למשהו מת.

ערך מוחזר של הפונקציה: 0 אם הצלחנו ו-1 בכישלון.

לסיכום: אם נרצה להכנס לקטע קריטי בקוד נעשה `sem_op=-1`, אם נרצה לצאת נעשה `sem_op=1`.

```
#include <sys/sem.h>
int semctl(int semid, int semnum, int cmd, semun arg)
```

3. **Semctl**: הפונקציה הנ"ל משמשת לביצוע פעולות ניהול – אתחול ערכים,

קבלת מידע, ומחיקת הסמפור מהמערכת שמסיימים. הפרמטרים שלה:

- `Semid`: מזהה הסמפור.
- `Semnum`: מס' הסמפור בסט עליו אנחנו פועלים. בפעולות שמשפיעות על כל הסט נתעלם מזה.
- `Cmd`: הפקודה.
- `Arg`: איחוד נתונים שמעבירים לפונקציה או מקבלים ממנה (תלוי בפקודה).

ישנן מס' פקודות `cmd` נפוצות –

- `IPC_STAT`: "תביא לי דוח מצב" – שולפת מידע על הסמפור (בעלים, הרשאות וכו') לתוך המבנה ב-`ARG`.
- `IPC_SET`: "עדכן הגדרות". מאפשר לשנות הרשאות גישה של סמפור קיים.
- `IPC_RMID`: מוחקת את הסמפור מהמערכת. חשוב: אם לא נסגור את הסמפור בסיום – הוא יישאר בזיכרון מערכת ההפעלה גם אחרי שהתוכנית תגמר.
- `SETVAL`: משמשת לקבוע את הערך ההתחלתי של הסמפור (למשל כ-1, שיתפקד כמנעול).

ערך מוחזר של הפונקציה: -1 ככישלון ואחרת ערך אי שלילי כהצלחה.

ישנה בעיה: לפעמים צריך להגדיר את `semun` בעצמנו. לפי התקן של לינוקס, הוא לא תמיד מוגדר בתוך הספרייה. אם לא נעתיק את הסטרוקט לקוד שלנו – הקומפיילר לא יזהה את הסטרוקט. לכן תמיד נעתיק את קטע הקוד הבא (משמאל):

```
union semun {
    int val; // value for SETVAL
    struct semid_ds *buf; // pointer to struct semid_ds for IPC_STAT, IPC_SET
    unsigned short *array; // array for GETALL, SETALL
};
struct semid_ds {
    struct ipc_perm sem_perm;
    unsigned short sem_nsems;
    time_t sem_otime; // last time of semop
    time_t sem_ctime; // last time of semctl
};
struct ipc_perm {
    key_t key;
    ushort uid; /* owner uid and gid */
    ushort gid;
    .
    ushort mode;
    ...
};
```

השוואה בין semaphore vs mutex:

Semaphore	Mutex	
Signaling mechanism	Locking mechanism	מטרה
בין תהליכים/threads	לשימוש באותו תהליך (ע"י threads)	ייעוד
ניתן להשתמש גם במונים (לאפשר כמה גישות לאותו משאב לדוגמא)	נעול/לא נעול	מצבים

למרות שמיוטקס וסמפור דומים – יש ביניהם הבדלים משמעותיים.

- מטרה: המטרה של מיוטקס היא לשמש כמנעול – ולהבטיח בלעדיות. בעוד המטרה של סמפור הוא לעבוד בשיטה של איתות - תהליך אחד יכול לאותת לתהליך אחר שהוא יכול להמשיך, הוא לא בהכרח מנעול אלא יותר כמו רמזור.
- ייעוד: הסמפור משמש לסנכרון בין תהליכים שונים או תרדים. הוא כלי חזק שמוכר ע"י מערכת ההפעלה כולה, בעוד המיוטקס משמש לרוב בתוך אותו תהליך (בין תרדים שחולקים אותו זיכרון). הוא קל ומהיר יותר.
- מצבים: במיוטקס יש שני מצבים בלבד – נעול או לא, בסמפור ייתכן מונה שמאפשר לשלושה (למשל) משאבים להכנס למערכת.
- כמו כן, במיוטקס יש מושג של בעלות – רק מי שנעל יכול לשחרר. בסמפור, תהליך א' יכול להיות בהמתנה וב' יעשה לו איתות שישחרר אותו.

תרגול 7

Signal: מדובר במכניזם שבעזרתו תהליכים מודעים על מאורעות שהתרחשו במערכת. איתות (signal) הוא מאורע אסינכרוני – כלומר: תהליך צריך להיות מוכן לקבל איתות בכל שלב של ריצתו. הסיגנל לא חוסם את התהליך שרץ – וקורא במקביל מאחורי הקלעים עד שהמקבל תופס את הסיגנל.

- הן מערכת ההפעלה, הן תהליך אחר והן התהליך עצמו – יכולים לאותת לתהליך.
- מדוע שנרצה לשלוח אותות? נרצה לשנות סטטוס של תהליך (לסיים תהליך SIGKILL, לעצור תהליך SIGSTOP, להמשיך תהליך שנעצר SIGCONT – כל מה שקורה במחזור החיים של תהליך), נרצה ליידע תהליך על בעיה (חריגה בזיכרון SIGSEGV, שגיאה אריתמטית – למשל חלוקה באפס SIGFPE), וכן עבור תקשורת בין תהליכים (SIGUSER1, SIGUSER2).
- בלינוקס יש סה"כ 64 סיגנלים. זה דומה לenum: ניתן לקרוא להם בשמם, וכן לכל סיגנל יש גם value מסויים. למשל: עבור SIGFPE ה-value=8.
- לכל סיגנל יש סוג פעולה – Action:
 1. Term: סיום התהליך
 2. Ign: להתעלם מהאיתות.
 3. Core: יצירת core dump file וסיום התהליך.
 4. Stop: עצירת התהליך.
 5. Cont: המשכת התהליך אם היה במצב עצור.

מה קורה כשמקבלים איתות? ישנן כמה אפשרויות –

1. לבצע את פעולת ברירת המחדל – SIG_DFL
2. להתעלם ממנו – ישנם שני איתות שאסור להתעלם מהם – SIGSTOP, SIGKILL.
3. לכתוב פונקציה שמטפלת בו.

אם כן, כיצד מגדירים פונקציית טיפול?

```
#include <signal.h>
```

```
typedef void (*sighandler_t)(int);
```

```
sighandler_t signal(int signum, sighandler_t handler)
```

פונקציה מסוג void, שלא מחזירה כלום, מקבלת את מספר הסיגנל, ערך שלם, וכן מקבלת את sighandler_t handler (רפרנס לפונקציית הטיפול). כלומר אכן כותבים פונקציית טיפול שמזמנת פונקציה אחרת. כלומר הפונקציה מחזירה מצביע לפונקציה הקודמת שהייתה רשומה לטפל בסיגנל המסוים. כאשר קוראים לפונקציה signal עם אותו מספר סיגנל (signum) אך עם פונקציית טיפול (handler) חדשה, מערכת ההפעלה פשוט מחליפה את הרישום הישן בחדש. מרגע זה והלאה, בכל פעם שיגיע הסיגנל, המערכת תפעיל את הפונקציה החדשה שהגדרנו.

אם יש שגיאה, הפונקציה תחזיר SINGERR.

משמאל, רשימת כל 64 הסיגנלים:

```
yaaron@GuyHP17:~$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL    10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO      30) SIGPWR
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```


הפונקציה Kill:

```
#include <sys/types.h>
#include <signal.h>
int kill(pid_t pid, int sig)
```

הפונקציה שולחת איתות לתהליך / קבוצת תהליכים. ערך מוחזר: 0 בהצלחה, 1- בכישלון.

- אם pid > 0 אזי ישלח איתות לתהליך שהה process id שלו הוא pid.
 - אם pid = 0 שולחים לכל התהליכים שהה process group id שלהם הוא pid.
 - אם pid = -1 שולחים איתות לכל התהליכים חוץ pid init.
- דוגמה טובה לשימוש באיתותים – ניתן לסמלץ wait באמצעות סיגנלים.

הפונקציה Pause:

pause

```
#include <unistd.h>
int pause(void)
```

כאשר תהליך רוצה להיות במצב "הפסקה" עד שהוא מקבל איתות מתהליך אחר נשתמש בקריאת מערכת הנ"ל. הפונקציה חוזרת (pausen משתחרר) רק כאשר פונקציית signal handler תפסה איתות וגם סיימה את פעולתה. ערך ההחזרה הוא 1-.

```
int main (void)
```

```
{
    printf("waiting for SIGCHLD\n");
    pause();
    printf("waiting for SIGCHLD again\n");
    pause();
    printf("I'm done\n");
    return 0;
}
```

הניחו שקיים תהליך כלשהו השולח ברצף סיגנלים מסוג SIGCHLD שפעולת ברירת המחדל שלו היא ignore לתהליך המריץ את התוכנית הנתונה. מה יודפס על המסך כתוצאה מריצת התוכנית?

שאלה. נתבונן בקטע הקוד משמאל:

פתרון: יודפס רק פעם אחת בדיוק. מדוע? לאחר ההדפסה הראשונה, התהליך נכנס למצב "הפסקה" עד שיקבל איתות כלשהו שהוגדר עבורו טיפול על ידי התהליך. רק במקרה זה יתעורר ממצב ה"הפסקה" וממשיך לרוץ. אבל מכיוון שהתהליך לא הגדיר טיפול באיתות ה-SIGCHLD הוא לא יתעורר עקב הגעת איתות זה.

הפונקציה alarm:

alarm

```
#include <unistd.h>
unsigned int alarm(unsigned int seconds)
```

תהליך יכול לבקש שישלחו אליו איתות מסוג SIGALRM בעוד מספר שניות. שימושי אם נרצה להגביל פעולה בזמן מסויים (ואחריה זה כבר לא רלוונטי וכו'). כל קריאה לalarm מבטלת את הalarm הקודם.

ערך int ש alarm מחזירה – מס' השניות שחסרות עד שהalarm היה צריך לקרות.

הפונקציה raise:

raise

```
#include <signal.h>
int raise(int sig)
```

שליחת סיגנל ע"י התהליך לעצמו. Sig הוא הסיגנל אותו נרצה לשלוח. ערך חוזר: 9 בהצלחה, ערך אחר מאפס בכישלון.

בקרת תהליכים:

על מנת להציג מידע על התהליכים שרצים כרגע במערכת ניתן להשתמש בפקודה `ps [options]`: הפקודה נותנת צילום מצב של מה שקורה כרגע במחשב. בפרט – את הpid של כל תהליך, הטרמינל TTY שכל תהליך מחובר אליו, כמה זמן מעבד התהליך צרך בפועל מאז שהתחיל, שם התוכנית / קובץ שהריצו.

Foreground vs Background

- **Foreground Job**: זה התהליך ש-"תופס" את הטרמינל. כל מה שמקלידים במקלדת הולך אליו, וכל מה שהוא מדפיס מופיע על המסך. רק תהליך אחד יכול להיות ב-Foreground בכל רגע נתון.
- **Background Job**: תהליך שרץ "מאחורי הקלעים". הוא לא קשוב למקלדת, אבל הוא ממשיך לבצע את הפעולות שלו.

איך שולטים עליהם?

- **דרך א': הדרך המהירה (&)** כשמריצים תוכנית, פשוט מוסיפים `&` בסוף השורה. `./a.out &`: התוכנית תרוץ ישר ברקע, והטרמינל יישאר פנוי לך לכתוב פקודות נוספות.
 - **דרך ב': העברה לרקע תוך כדי ריצה (תהליך של שני שלבים)** נניח שהרצת תוכנית, היא רצה ב-`Foreground`-ופתאום נזכרת שאתה רוצה להמשיך להשתמש בטרמינל בזמן שהיא רצה:
 - **CTRL+Z**: זה שולח סיגנל שנקרא `SIGTSTP` (עצירה). התוכנית נכנסת למצב "קפוא" (`Stopped`). היא לא נסגרת, היא פשוט עוצרת את הפעולה שלה.
1. **bg (Background)**: זו פקודה שאומרת למערכת: "קחי את התהליך האחרון שעצרתי, ותמשיכי להריץ אותו, אבל הפעם ברקע."

למה זה שימושי?

זה קריטי כשמריצים פעולות ארוכות (כמו קומפילציה גדולה, העתקת קבצים או הרצת שרת). אתה לא רוצה שהטרמינל יהיה "תפוס" ושלא תוכל לעשות שום דבר אחר בזמן שהתוכנית עובדת. השימוש ב-`&` או ב-`bg`-מאפשר לך לשחרר את הטרמינל ולהמשיך לעבוד במקביל.

סיפורים של סוף תרגול:

לפי עינת, המתרגלים ביקשו וקיבלו אישור מדודי שתהיה במבחן שאלת בונוס של 3 נקודות על הסיפורים שמופיעים בסוף כל תרגול ולכן הם מרוכזים פה על מנת הנוחות שלנו:

- **תרגול 1:** ב-2015, מרקו מרסאלה פרסם בפורום טכני שהוא בטעות הריץ פקודת `rm -rf` עם הרחבת משתנה שגויה על שרתי חברת האחסון שלו – ומחק את כל הנתונים של 1,535 אתרים, כולל הגיבויים שהיו מעוגנים לאותה מערכת קבצים. העסק נהרס בפקודה אחת. זו הסיבה שחשוב להבין איך ה-shell מרחיב משתנים, איך `rm` עובד ברמת ה-syscall, ולמה `rm` לא באמת מוחק נתונים (הוא מבצע `unlink` ל-inode). להכיר את מערכת ההפעלה זה להבין מה פקודה באמת עושה – לא מה שאתם חושבים שהיא עושה. מה המסר? **פקודה אחת שגויה יכולה למחוק חברה שלמה.**
- **תרגול 2:** בספטמבר 2014 התגלה באג בן 25 שנה ב-Bash עצמו – אותו shell שלמדתם היום לכתוב בו סקריפטים. הבאג, שנקרא Shellshock (CVE-2014-6271), ניצל את האופן שבו Bash מטפל במשתני סביבה והגדרות פונקציות. תוקף יכול היה להזריק פקודות שרירותיות דרך משתני סביבה מעוצבים במיוחד. מכיוון ש-Bash נמצא בכל מקום – שרתי ווב (CGI), SSH, לקוחות DHCP – הבאג הזה חשף מאות מיליוני מערכות ברחבי העולם. הבאג התקיים מאז Bash גרסה 1.03 ב-1989. לקח 25 שנה עד שמישהו שם לב. ה-shell הוא לא רק כלי עבודה, הוא משטח תקיפה קריטי. באג אחד ב-Bash השבית מיליוני מערכות והזכיר לעולם שגם בסקריפטים פשוטים – *With great power comes great responsibility!*
- **תרגול 3:** ה-fork bomb הקלאסי: `{&|: }{&|: }`; הוא רק 13 תווים – והוא יכול להפיל כמעט כל מערכת Linux לא מוגנת תוך שניות. זוהי פונקציה שקוראת לעצמה פעמיים, יוצרת צמיחה מעריכית של תהליכים. fork bomb בודד יכול ליצור מעל מיליון תהליכים בפחות מ-60 שניות. בתקרית אמיתית, pull request זדוני ב-GitHub Actions הכיל fork bomb בתוך סוויט הבדיקות, מה שהפיל את כל ה-runner host והשפיע על builds אחרים. אז מה המסר? גם פקודה שנראית תמימה יכולה להוריד שרתים שלמים ברגע ולכן חשוב ולפעול בזהירות ובצורה בטוחה, מי שמנסה לבלוע יותר מדי תהליכים "במזלג" אחד – עשוי לגרום למערכת כולה להיחנק (או במילים אחרות, תפסת מרובה - לא תפסת).
- **תרגול 4:** בין 1985 ל-1987, מכונת הקרנות Therac-25 העבירה מנות קרינה מסיביות לפחות ל-6 מטופלים – חלקם קיבלו מאות מונים מהמינון המתוכנן. שלושה מטופלים מתו. הסיבה? Race condition. לתוכנת המכונה היו שגרות מקבילות שחלקן משתנה בלי סנכרון נכון (בלי mutex!). אם מפעיל הקליד פקודות מהר מספיק במהלך חלון הגדרה של 8 שניות, התוכנה נכנסה למצב לא עקבי – הפעילה את הקרן בעוצמה מלאה בלי מפזר הבטיחות. החלק המפחיד: אותו באג בדיוק התקיים בדגם הקודם (Therac-20), אבל נעילות בטיחות חומרתיות הסתירו אותו. כשה-Therac-25 החליף נעילות חומרה בבדיקות תוכנה בלבד, ה-race condition הסמוי הפך קטלני. אז מה המסר? שלושה אנשים מתו בגלל mutex חסר. המושגים שלמדתם היום הם לא אקדמיים – הם מצילים חיים.
- **תרגול 5:** הפילוסופיה של Unix ש"הכל זה קובץ" אומרת ש-file descriptors הם לא רק לקריאת קבצי טקסט. ב-Linux, `/dev/null` הוא קובץ שזורק הכל. `/dev/random` הוא קובץ שמייצר בתים אקראיים. `/proc/cpuinfo` הוא קובץ שמראה מידע על ה-CPU – אבל הוא נוצר בזמן אמת על ידי הקרנל; אין קובץ אמיתי בדיסק. הקריאה `dup()` שלמדתם היום היא האופן שבו ה-shell מממש `/O redirection`: כשאתם כותבים `ls > output.txt`, ה-shell קורא ל-`dup2()` כדי להחליף את ה-file descriptor של `stdout` באחד שמצביע לקובץ. אז מה המסר?
- בדיוק כמו ב-Linux, הכוח האמיתי לא נמצא במורכבות של הכלים, אלא בפשטות של הממשק. כשאנחנו לומדים לזקק את המציאות לשפה אחת אחידה, אנחנו מפסיקים לנהל בעיות נקודתיות ומתחילים לשלוט בארכיטקטורה של הפתרון.
- **תרגול 6:** ביולי 1997, רובר ה-Mars Pathfinder של NASA התחיל לחוות איפוס מערכת מסתוריים ימים אחרי הנחיתה על מאדים. הסיבה? היפוך עדיפויות (priority inversion) – בעיית סנכרון. משימת איסוף נתוני מזג אוויר בעדיפות נמוכה החזיקה mutex על אפיק משותף. משימת ניהול האפיק בעדיפות גבוהה הייתה צריכה את ה-mutex ונחסמה. אבל משימת תקשורת בעדיפות בינונית המשיכה לעקוף את המשימה בעדיפות הנמוכה, מנעה ממנה לשחרר את ה-mutex. טיימר השמירה

סיכום מערכות הפעלה | © גיא יער-און

- (watchdog) שם לב שהמשימה בעדיפות הגבוהה לא רצה, הבין שמשוהו השתבש, ואילץ איפוס מערכת מלא – שמחק את כל הנתונים שנאספו. מהנדסי JPL שחזרו את הבאג על רפליקה מדויקת בכדור הארץ אחרי 18 שעות בדיקות. התיקון? הפעלת priority inheritance על ה-mutex – דגל boolean בודד ב-VxWorks, שהועלה למאדים כתוכנית C קצרה דרך interpreter שהיה מותקן על החללית. החללית הריצה VxWorks RTOS עם semaphores ו-mutexes. שלושת החוקרים שזיהו במקור את בעיית היפוך העדיפויות והציעו את פתרון ה-priority inheritance שא, להוצ'קי, ורג'קומאר מ (CMU-היו כולם בקהל כשהסיפור סופר בכנס IEEE. אז מה המסר? ניהול משאבים נכון הוא ההבדל בין הצלחה על מאדים לקריסה בייצור וזמן הוא המשאב הכי יקר שלנו בחיים... אז וודאו תמיד שסדרי העדיפויות שלכם נכונים!
- **תרגול 7:** בכל פעם שאתם לוחצים Ctrl+C בטרמינל, אתם שולחים SIGINT (סיגנל 2) – בקשה ידידותית לעצור. אבל בעולם ה-Linux, המלך האמיתי הוא SIGKILL (סיגנל 9). זהו הסיגנל היחיד שהתהליך שלכם לא יכול לתפוס, לחסום או להתעלם ממנו. אין signal handler שיציל אתכם. הקרנל פשוט מוחק את התהליך מהזיכרון. אבל יש מלכוד (והוא מרתק): אפילו SIGKILL יכול להיכשל. אם תהליך תקוע ב-Uninterruptible Sleep (מצב D ב-ps, לרוב בגלל המתנה לדרייבר או דיסק תקוע), הקרנל לא יכול להעיר אותו כדי להרוג אותו. התהליך הופך ל"זומבי חסין אש". התייעוד הרשמי של Linux אומר על זה משפט נדיר: "זה מהווה באג במערכת ההפעלה שצריך לדווח עליו". אז מה המסר?
- ארכיטקטורה טובה – בקוד, בצוות או בחיים – נמדדת ביכולת לעשות Graceful Shutdown לאיתותים קטנים, לפני שהמציאות שולחת SIGKILL.